

Share Your Experiences	
Geometric Algorithms	
Problems based on Pattern Printing	
Problems based on Lines	
How to check if two given line segments intersect?	
Given n line segments, find if any two segments intersect	
Klee's Algorithm (Length Of Union Of Segments of a line)	
Count maximum points on same line	
Minimum lines to cover all points	
Represent a given set of points by the best possible straight line	
Program to find line passing through 2 Points	
Reflection of a point about a line in C++	
Program to find the mid-point of a line	
Sum of Manhattan distances between all pairs of points	
Program to check if three points are collinear	
Problems based on Triangles	
Problems based on Rectangle, Square and Circle	
Problems based on 3D Objects	
Problems based on Quadrilateral	
Problems based on Polygon and Convex Hull	
Find Simple Closed Path for a given set of points	
Closest Pair of Points using Divide and Conquer algorithm	
Optimum location of point to minimize total distance	
Count of parallelograms in a plane	
Rotation of a point about another point	
Finding the vertex, focus and directrix of a parabola	
Find mirror image of a point in 2-D plane	
Equable Shapes	
DSA to Development	Course

Given n line segments, find if any two segments intersect

Last Updated : 19 Sep, 2022

Given n line segments (p1, q1], (p2, q2), ... (pn, qn), find if the given line segments intersect with each other or not.

Example:

Input: $\{[1, 5], [4, 5]\}, \{[2, 5], [10, 11]\}, \{[3, 2], [10, 3]\}, \{[6, 4], [9, 4]\}, \{[7, 1], [8, 1]\}$
Output: 2

Explanation: Lines $\{[1, 5], [4, 5]\}, \{[2, 5], [10, 11]\}$ are intersecting, as well as $\{[2, 5], [10, 11]\}, \{[3, 2], [10, 3]\}$.

Recommended Problem

Check If two Line segments Intersect

Companies: [Snapdeal](#) [Adobe](#) [+1 more](#) Medium 24.51% 47.4K

Show Topics

We have discussed the problem to detect if [two given line segments intersect or not](#). In this post, we extend the problem. Here we are given n line segments and we need to find out if any two line segments intersect or not.

Naive Algorithm A naive solution to solve this problem is to check every pair of lines and check if the pair intersects or not. [We can check two line segments in O\(1\) time](#). Therefore, this approach takes $O(n^2)$.

Sweep Line Algorithm We can solve this problem in $O(n \log n)$ time using Sweep Line Algorithm. The algorithm first sorts the end points along the x axis from left to right; then it passes a vertical line through all points from left to right and checks for intersections. Following are detailed steps.

- 1) Let there be n given lines. There must be 2n end points to represent the n lines. Sort all points according to x coordinates. While sorting maintain a flag to indicate whether this point is left point of its line or right point.
- 2) Start from the leftmost point. Do following for every point
 - a) If the current point is a left point of its line segment, check for intersection of its line segment with the segments just above and below it. And add its line to active line segments (line segments for which left end point is seen, but right end point is not seen yet). Note that we consider only those neighbors which are still active.
 - b) If the current point is a right point, remove its line segment from active list and check whether its two active neighbors (points just above and below) intersect with each other.

The step 2 is like passing a vertical line from all points starting from the leftmost point to the rightmost point. That is why this algorithm is called Sweep Line Algorithm. The Sweep Line technique is useful in many other geometric algorithms like calculating the 2D Voronoi diagram

What data structures should be used for efficient implementation?

In step 2, we need to store all active line segments. We need to do following operations efficiently:

- a) Insert a new line segment
 - b) Delete a line segment
 - c) Find predecessor and successor according to y coordinate values
- The obvious choice for above operations is Self-Balancing Binary Search Tree like AVL Tree, Red Black Tree. With a Self-Balancing BST, we can do all of the above operations in $O(\log n)$ time.
- Also, in step 1, instead of sorting, we can use min heap data structure. Building a min heap takes $O(n)$ time and every extract min operation takes $O(\log n)$ time (See [this](#)).

PseudoCode:

The following pseudocode doesn't use heap. It simply sort the array.

```
sweepLineIntersection(points[0..2n-1]):
1. Sort Points[] from left to right (according to x coordinate)

2. Create an empty Self-Balancing BST T. It will contain
   all active line segments ordered by y coordinate.

// Process all 2n points
3. for i = 0 to 2n-1
    // If this point is left end of its line
    if (points[i].isLeft)
        T.insert(points[i].line()) // Insert into the tree

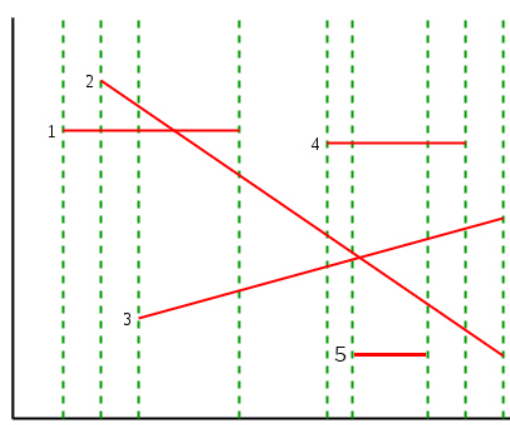
    // Check if this point intersects with its predecessor and successor
    if ( ! doIntersect(points[i].line(), T.pred(points[i].line()) ) )
        return true
    if ( ! doIntersect(points[i].line(), T.succ(points[i].line()) ) )
        return true

    else // If it's a right end of its line
        // Check if its predecessor and successor intersect with each other
        if ( ! doIntersect(T.pred(points[i].line()), T.succ(points[i].line()) ) )
            return true
        T.delete(points[i].line()) // Delete from tree

4. return False
```

Example:

Let us consider the following example taken from [here](#). There are 5 line segments **1, 2, 3, 4** and **5**. The dotted green lines show sweep lines.



Sweep Lines

Following are steps followed by the algorithm. All points from left to right are processed one by one. We maintain a self-balancing binary search tree.

Left end point of line segment 1 is processed: 1 is inserted into the Tree. The tree contains 1. No intersection.

Left end point of line segment 2 is processed: Intersection of 1 and 2 is checked. 2 is inserted into the Tree. Intersection of 1&2 Found ("Note that the above pseudocode returns at this point"). We can continue from here to report all intersection points. The tree contains **1, 2**.

Left end point of line segment 3 is processed: Intersection of 3 with 1 is checked. No intersection. 3 is inserted into the Tree. The tree contains **2, 1, 3**.

Right end point of line segment 1 is processed: 1 is deleted from the Tree. Intersection of 2 and 3 is checked. Intersection of 2 and 3 is reported. The tree contains **2, 3**.

Left end point of line segment 4 is processed: Intersections of line 4 with lines 2 and 3 are checked. No intersection. 4 is inserted into the Tree. The tree contains **2, 4, 3**.

Left end point of line segment 5 is processed: Intersection of 5 with 3 is checked. No intersection. 5 is inserted into the Tree. The tree contains **2, 4, 3, 5**.

Right end point of line segment 5 is processed: 5 is deleted from the Tree. The tree contains **2, 4, 3**.

Right end point of line segment 4 is processed: 4 is deleted from the Tree. The tree contains **2, 4, 3**. Intersection of 2 with 3 is checked. Intersection of 2 with 3 is reported (Note that the intersection of 2 and 3 is reported again. We can add some logic to check for duplicates). The tree contains **2, 3**.

Right end point of line segment 2 and 3 are processed: Both are deleted from tree and tree becomes empty.

C++14

```
#include <bits/stdc++.h>
using namespace std;

// A point in 2D plane
struct Point
{
    int x, y;
};

// A line segment with left as Point
// with smaller x value and right with
// larger x value.
struct Segment
{
    Point left, right;
};

// An event for sweep line algorithm
// An event has a point, the position
// of point (whether left or right) and
// index of point in the original input
// array of segments.
struct Event {
    int x, y;
    bool isLeft;
    int index;
    Event(int x, int y, bool l, int i) : x(x), y(y), isLeft(l), index(i) {}

    // This is for maintaining the order in set.
    bool operator<(const Event& e) const {
        if(y==e.y)return x<e.x;
        return y < e.y;
    }
};

// Given three collinear points p, q, r, the function checks if
// point q lies on line segment 'pr'
bool onSegment(Point p, Point q, Point r)
{
    if (q.x <= max(p.x, r.x) && q.x >= min(p.x, r.x) &&
        q.y <= max(p.y, r.y) && q.y >= min(p.y, r.y))
        return true;

    return false;
}

// To find orientation of ordered triplet (p, q, r).
// The function returns following values
// 0 -> p, q and r are collinear
// 1 -> Clockwise
// 2 -> Counterclockwise
int orientation(Point p, Point q, Point r)
{
    // See https://www.geeksforgeeks.org/orientation-3-ordered-points/
    // for details of below formula.
    int val = (q.y - p.y) * (r.x - q.x) -
            (q.x - p.x) * (r.y - q.y);

    if (val == 0) return 0; // collinear
    return (val > 0) ? 1 : 2; // clock or counterclock wise
}

// The main function that returns true if line segment 'p1q1'
// and 'p2q2' intersect.
bool doIntersect(Segment s1, Segment s2)
{
    Point p1 = s1.left, q1 = s1.right, p2 = s2.left, q2 = s2.right;

    // Find the four orientations needed for general and
    // special cases
    int o1 = orientation(p1, q1, p2);
    int o2 = orientation(p1, q1, q2);
    int o3 = orientation(p2, q2, p1);
    int o4 = orientation(p2, q2, q1);

    // General case
    if (o1 != o2 && o3 != o4)
        return true;

    // Special Cases
    // p1, q1 and p2 are collinear and p2 lies on segment p1q1
    if (o1 == 0 && onSegment(p1, p2, q1)) return true;

    // p1, q1 and q2 are collinear and q2 lies on segment p1q1
    if (o2 == 0 && onSegment(p1, q2, q1)) return true;

    // p2, q2 and p1 are collinear and p1 lies on segment p2q2
    if (o3 == 0 && onSegment(p2, p1, q1)) return true;

    // p2, q2 and q1 are collinear and q1 lies on segment p2q2
    if (o4 == 0 && onSegment(p2, q1, q2)) return true;

    return false; // Doesn't fall in any of the above cases
}

// Find predecessor of iterator in s.
set<Event>::iterator pred(set<Event> &s, set<Event>::iterator it) {
    return it == s.begin() ? s.end() : --it;
}

// Find successor of iterator in s.
set<Event>::iterator succ(set<Event> &s, set<Event>::iterator it) {
    return ++it;
}
```


Given n line segments, find if any two segments intersect - GeeksforGeeks

Given three points on the X-axis which denotes the center of three line segments. The length of the line segment is also given as L. The task is to move the center of the given line segments by a distance of K to maximize the length of...

5 min read

Number of ways to choose K intersecting line segments on X-axis

Given an array arr[] of line segments on the x-axis and an integer K, the task is to calculate the number of ways to choose the K line-segments such that they do intersect at any point. Since the answer can be large, print it to modulo 10^9+7....

10 min read

Article Tags : DSA Geometric Geometric-Lines

Practice Tags : Geometric



Corporate & Communications Address: A-143, 3th Floor, Sovereign Corporate Tower, Sector-136, Noida, Uttar Pradesh (201306)
Registered Address: K. 001, Tower K, Gulshan Vivante Apartment, Sector-137, Noida, Gautam Buddha Nagar, Uttar Pradesh, 201305



@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved

Company

About Us
Legal
Careers
In Media
Contact Us
Advertise with us
GFG Corporate Solution
Placement Training Program

Python Tutorial

Python Programming Examples
Django Tutorial
Python Projects
Python Tutorials
Web Scraping
OpenCV Tutorial
Python Interview Question

Databases

SQL
MySQL
PostgreSQL
PL/SQL
MongoDB

Explore

Job-A-Thon Hiring Challenge
Hack-A-Thon
GFG Weekly Contest
Offline Classes (Delhi/NCR)
DSA in Java/C++
Master System Design
Master C++
GeeksforGeeks Videos
Geeks Community

Computer Science

GATE CS Notes
Operating Systems
Computer Network
Database Management System
Software Engineering
Digital Logic Design
Engineering Maths

Preparation Corner

Company-Wise Recruitment Process
Resume Templates
Aptitude Preparation
Puzzles
Company-Wise Preparation
Companies
Colleges

Languages

Python
Java
C++
PHP
GoLang
SQL
R Language
Android Tutorial

DevOps

Git
AWS
Docker
Kubernetes
Azure
GCP
DevOps Roadmap

Competitive Exams

JEE Advanced
UAC NET
UPSC
SSC CGL
SBI PO
SBI Clerk
IBPS PO
IBPS Clerk

DSA

Data Structures
Algorithms
DSA for Beginners
Basic DSA Problems
DSA Roadmap
DSA Interview Questions
Competitive Programming

System Design

High Level Design
Low Level Design
UML Diagrams
Interview Guide
Design Patterns
OOD
System Design Bootcamp
Interview Questions

More Tutorials

Software Development
Software Testing
Product Management
Project Management
Linux
Excel
All Cheat Sheets
Recent Articles

Data Science & ML

Data Science With Python
Data Science For Beginner
Machine Learning
ML Maths
Data Visualisation
Pandas
NumPy
NLP
Deep Learning

School Subjects

Mathematics
Physics
Chemistry
Biology
Social Science
English Grammar

Free Online Tools

Typing Test
Image Editor
Code Formatters
Code Converters
Currency Converter
Random Number Generator
Random Password Generator

Web Technologies

HTML
CSS
JavaScript
TypeScript
ReactJS
NextJS
NodeJS
Bootstrap
Tailwind CSS

Commerce

Accountancy
Business Studies
Economics
Management
HR Management
Finance
Income Tax

Write & Earn

Write an Article
Improve an Article
Pick Topics to Write
Share your Experiences
Internships